

Day 1 Learning

#JDK, JRE, JVM

JDK is Java Development Kit, which contains JRE along with tools & libraries which are used to execute and compile a java program. It is Platform Dependent. You download a specific JDK for Windows, Mac, or Linux.

Which tools it contains:

- Javac (Compiler)
- Java (Loader/Launcher)
- Jdb (Debugger)
- Jar (Archiver)

JRE is Java Runtime Environment which contains JVM and provides an environment to run the java program. JVM resides inside the JRE. It contains the Class Libraries (like rt.jar or the newer module system) that your code needs at runtime. If you only want to run a Java app (not code it), you only need the JRE.

JVM is Java Virtual Machine which contains the interpreter and Just In Time (JIT) Compiler.

- The Interpreter reads bytecode and executes it immediately (slower).
- The JIT Compiler identifies "hot spots" (code executed frequently) and compiles them into Native Machine Code for direct hardware execution (much faster).

The JVM is what makes Java Platform Independent. It follows the "Write Once, Run Anywhere" (WORA) principle because it translates the same .class file for any OS.

#JVM Architecture

The Java Virtual Machine (JVM) is the engine that provides a runtime environment to drive Java applications. It is responsible for converting bytecode into machine-specific code. The architecture is primarily divided into three subsystems: the Class Loader Subsystem, Runtime Data Areas, and the Execution Engine.

Class Loader Subsystem:

The Class Loader Subsystem is responsible for loading, linking, and initializing the class files. It follows the Delegation Hierarchy Algorithm to load class files.

1. Loading:

Loading is the process of finding the binary representation of a class or interface type with a particular name and creating a class or interface from that representation. There are three main types of Class Loaders:

- **Bootstrap Class Loader:** This is the parent of all class loaders. It is responsible for loading standard JDK internal classes located in the bootstrap path (typically `rt.jar` and other core libraries in the `lib` directory). In modern Java versions, it loads the `java.base` module and other essential modules. It is implemented in native languages like C or C++.
- **Extension Class Loader (Platform Class Loader):** This is the child of the Bootstrap Class Loader. It loads classes from the JDK extension directories (usually `lib/ext`). In Java 9 and later, it is renamed to the Platform Class Loader and loads various Java SE platform APIs and non-core modules.
- **Application Class Loader (System Class Loader):** This is the child of the Extension/Platform Class Loader. It is responsible for loading classes from the application classpath, which is defined using the environment variable `CLASSPATH` or the `-cp` command-line option. This is the loader that loads the classes you write.

2. Linking:

Linking involves taking a class or interface and combining it into the runtime state of the JVM so that it can be executed. It involves three steps:

- **Verification:** This ensures the structural correctness of the bytecode. It checks if the code follows the rules of the Java Language Specification and does not violate JVM security. If verification fails, a `VerifyError` is thrown.
- **Preparation:** In this phase, the JVM allocates memory for class variables (static fields) and initializes those variables to their default values (e.g., 0 for int, null for objects, false for boolean). No code from the class is executed yet.
- **Resolution:** This is the process of replacing symbolic references in the constant pool with direct references (memory addresses). This can happen lazily (when the reference is first used).

3. Initialization

Initialization is the final phase of class loading. Here, the class's static initializers and static variable assignments are executed in the order they appear in the code. This is where the actual values you assigned to static variables are applied. A class is initialized only when it is first used, such as when a static method is called or an instance is created.

Runtime Data Areas

These are memory areas allocated by the JVM during program execution:

- **Method Area:** Stores class-level data, including the constant pool, field and method data, and the code for methods and constructors. It is shared among all threads.
- **Heap:** The area where all objects and their corresponding instance variables are stored. It is the main target of the Garbage Collector and is shared among all threads.
- **Stack:** Every thread has its own runtime stack. For every method call, a block called a Stack Frame is created, which stores local variables and intermediate results.
- **PC Registers:** Each thread has its own Program Counter Register to hold the address of the current JVM instruction being executed.

- Native Method Stack: Contains all the native methods used in the application.

Execution Engine

The Execution Engine executes the bytecode. It contains:

- Interpreter: Reads bytecode and executes it line by line.
- JIT Compiler: Compiles frequently used code (hotspots) into native machine code to improve performance.
- Garbage Collector: Automatically manages memory by deleting objects that are no longer reachable.

#Java Memory Management (JMM)

Java Memory Management is the process of allocating memory to objects and freeing it when those objects are no longer required. Efficient memory management is handled automatically by the JVM through the Garbage Collector, which helps prevent memory leaks and overflow errors. Understanding how memory is divided and managed is essential for writing high-performance applications.

1. Memory Structure (Heap vs. Stack)

The JVM divides memory into two primary sections: Stack memory and Heap memory.

- Stack Memory: Stack memory is used for the execution of a thread. It contains method-specific values that are short-lived, such as local variables and references to objects in the heap. Stack memory is thread-safe because each thread has its own stack. It follows a Last-In-First-Out (LIFO) order. When a method finishes execution, its corresponding stack frame is flushed.
- Heap Memory: Heap memory is used for the dynamic allocation of objects and JRE classes at runtime. All objects created using the "new" keyword are stored here. Unlike the stack, the heap is shared across all threads. It is further divided into generations to facilitate efficient garbage collection.

2. Heap Memory Generations

The heap is organized into different generations based on the age of the objects:

- Young Generation: This is where all new objects are created. It is subdivided into three parts: Eden Space, Survivor Space 0 (S0), and Survivor Space 1 (S1). When the Eden space fills up, a Minor Garbage Collection occurs. Surviving objects are moved to the survivor spaces.
- Old (Tenured) Generation: Objects that have survived multiple rounds of Minor Garbage Collection in the Young Generation are eventually moved here. This area is much larger than the Young Generation. When it fills up, a Major Garbage Collection occurs.
- Permanent Generation (Metaspace): In older versions of Java, this was the PermGen. Since Java 8, it has been replaced by Metaspace. It stores class metadata, static variables, and the method area. Unlike the heap, Metaspace grows automatically and uses native memory.

3. The Java Memory Model (Concurrency)

The JMM also defines how the Java runtime interacts with the computer's hardware memory. It acts as a set of rules for how different threads interact through memory.

- **Visibility:** This ensures that changes made by one thread to shared data are visible to other threads. Without proper synchronization or the use of the "volatile" keyword, a thread might keep a local copy of a variable in its CPU cache and not see the updated value in the main memory.
- **Atomicity:** This refers to operations that happen all at once or not at all. For example, reading a 64-bit long or double value might not be atomic on all platforms unless the variable is declared as volatile.
- **Instruction Reordering:** To improve performance, the JIT compiler or the CPU might reorder the sequence of instructions. The JMM establishes "happens-before" rules to ensure that reordering does not break the logic of multi-threaded code.

4. Common Memory Issues

- **StackOverflowError:** Occurs when the stack memory is full, typically due to deep or infinite recursion.
- **OutOfMemoryError:** Occurs when the heap memory is full and the Garbage Collector cannot free up any more space for new object allocation. This often happens if the application has a memory leak, where objects are still being referenced even though they are no longer needed.

#Reference Types in Java

In Java, not all object references are treated the same by the Garbage Collector. Reference types allow a developer to reach an object while still providing the Garbage Collector with the flexibility to reclaim memory if needed. There are four main types of references: Strong, Soft, Weak, and Phantom.

1. Strong Reference

A strong reference is the default type of reference in Java. Any object created using the "new" keyword is a strong reference. The Garbage Collector will never reclaim an object that has a strong reference pointing to it, even if the JVM is running out of memory. If memory becomes insufficient, the JVM will throw an OutOfMemoryError rather than deleting a strongly referenced object.

Example: `String s = new String("Hello");`

2. Soft Reference

A Soft Reference is used for objects that are memory-sensitive, such as caches. The Garbage Collector is not aggressive with soft references. It will only reclaim these objects if the JVM absolutely needs memory to avoid an OutOfMemoryError. If the JVM has enough memory, softly referenced objects will stay in the heap.

Use Case: Implementing memory-sensitive caches where you want to keep data as long as possible but are willing to let it go if the system is under memory pressure.

3. Weak Reference

A Weak Reference is even less persistent than a soft reference. When the Garbage Collector runs, it will immediately reclaim/delete any object that is only held by weak references, regardless of whether memory is low or not. It does not wait for a memory crisis to act.

Use Case: WeakHashMap is a common example. It is useful for associating metadata with an object without preventing the object itself from being garbage collected when it is no longer used by the rest of the application.

4. Phantom Reference

A Phantom Reference is the weakest type of reference. An object that is phantomly reachable cannot be accessed through the reference directly; the get() method always returns null. Instead, phantom references are used to track when an object has been removed from memory.

Use Case: Performing flexible post-mortem cleanup actions. It is a safer and more efficient alternative to the deprecated finalize() method.

5. Reference Queue

Both Weak and Soft references can be registered with a Reference Queue. When the Garbage Collector determines that the reachability of an object has changed (e.g., it is about to be cleared), the reference object itself is appended to the queue. This allows the program to take specific actions once an object is ready for collection.

#Garbage Collection in Java

Garbage Collection (GC) is the process by which the JVM automatically identifies and deletes objects that are no longer being used by the application, thereby reclaiming heap memory. This prevents memory leaks and manual memory management errors.

1. The Generational Hypothesis

Most objects in Java are short-lived. Based on this, the Heap is divided into generations to optimize the collection process:

- Young Generation: Consists of Eden Space and two Survivor Spaces (S0 and S1). Most objects are created here.
- Old (Tenured) Generation: Stores long-lived objects that have survived multiple garbage collection cycles in the Young Generation.

2. Minor vs. Major vs. Full GC

- Minor GC: This occurs when the Eden space in the Young Generation fills up. It is frequent and very fast. During this process, unreachable objects are removed, and reachable objects are moved to a Survivor Space or promoted to the Old Generation.
- Major GC: This occurs when the Old Generation fills up. It is less frequent than Minor GC but takes longer because it involves a larger memory area.
- Full GC: This cleans the entire Heap, including both the Young and Old Generations, and sometimes the Metaspace. This is the most "expensive" operation in terms of performance and usually triggers a "Stop-the-World" event.

3. Core Garbage Collection Algorithms

The JVM uses several algorithms to identify and clear memory:

- Mark and Sweep:

Mark: The GC traverses the object graph starting from "GC Roots" (like local variables or static fields) and marks all reachable objects.

Sweep: The GC scans the heap and removes any objects that were not marked.

- Mark and Compact: Similar to Mark and Sweep, but after sweeping, it moves all remaining objects to one end of the memory block. This eliminates memory fragmentation, making it easier to allocate large objects later.
- Copying Algorithm: The memory is divided into two halves. Objects are created in one half. When it fills up, reachable objects are copied to the second half, and the first half is completely cleared. This is used primarily in the Young Generation (S0 and S1).

4. Types of Garbage Collectors

Developers can choose different GC implementations based on application needs:

- Serial GC: Uses a single thread for all GC operations. Best for small applications with low memory requirements.
- Parallel GC: Uses multiple threads for Young Generation collection. It is the default in many JVM versions and focuses on high throughput.
- G1 (Garbage First) GC: Designed for large heaps. It divides the heap into many small regions and prioritizes collecting regions that are mostly full of "garbage" first. It aims to provide predictable pause times.
- ZGC (Z Garbage Collector): A scalable, low-latency garbage collector designed to handle heaps ranging from a few megabytes to multiple terabytes with very short pause times (less than 1ms).

##Interview Questions:

1. Can you explain the difference between JDK, JRE, and JVM? Which one is platform-dependent?
2. If Java is platform-independent, why do we need different JDKs for Windows, Mac, and Linux?
3. Describe the path of a Java source file from `.java` to execution. Where do `javac` and the `Interpreter` come into play?
4. What are the main components of the JVM? (Be ready to talk about the Class Loader, Runtime Data Areas, and Execution Engine).
5. What is the "Delegation Hierarchy" in Class Loaders? What happens if two different class loaders try to load the same class?
6. What is the Just-In-Time (JIT) compiler, and how does it improve performance compared to a simple interpreter?
7. What is the main difference between Stack and Heap memory? Which one is thread-safe and why?
8. If I have a local variable that is an Object, where is the reference stored and where is the actual object stored?
9. What is Metaspace (formerly PermGen), and why was it moved to native memory in Java 8?
10. How does the `volatile` keyword relate to the Java Memory Model?
11. Explain the difference between a shallow copy and a deep copy. When would a shallow copy cause a bug in an application?
12. How would you implement a Deep Copy for a complex object containing multiple nested lists and objects?
13. Why is the `Cloneable` interface and `Object.clone()` generally considered "broken" or discouraged in the Java community?
14. When does an object become eligible for Garbage Collection? Can we force the GC to run?
15. Why is the Heap divided into Young and Old generations? (Explain the Generational Hypothesis).
16. What is the difference between "Mark and Sweep" and "Mark and Compact"? Why do we need the compacting step?
17. If you were building a low-latency trading application, which Garbage Collector would you choose (G1, ZGC, or Parallel) and why?
18. Where are static variables stored in memory?
19. Why can't we access non-static variables from a static method?
20. What is the order of execution for static blocks vs. instance blocks vs. constructors?
21. Can you explain the "Access Level" of the four modifiers? (Don't forget the **Default/Package-Private** one!).
22. If a class has a `protected` member, can a subclass in a *different* package access it? Can a non-subclass in the *same* package access it?
23. Why would you ever make a variable `private` and provide `public` getters/setters instead of just making the variable `public`?
24. Why can't a top-level class be declared as `private` or `protected`?
25. Why is the `main` method always `public static void`? What happens if you remove `static`?

26. What is the purpose of the `String[] args` parameter? Can we change the name from `args` to something else?
 27. Can we overload the `main` method in Java? If yes, will the JVM execute the overloaded version?
 28. What is "Variable Shadowing"? If a local variable has the same name as an instance variable, how do you access the instance variable inside that method?
 29. When is an instance variable destroyed compared to a local variable?
 30. Can a static method access an instance variable? Why or why not?
 31. Rank Strong, Soft, and Weak references in order of how likely they are to be cleared by the Garbage Collector.
 32. Exactly *when* does the GC clear a **Soft Reference** vs. a **Weak Reference**? (Hint: One is about memory pressure, the other is about the GC cycle).
 33. Why would you use a `WeakHashMap` instead of a regular `HashMap`?
 34. How can "Strong References" lead to memory leaks in a long-running Java application?
-
35. If I have an object that is pointed to by **both** a Strong Reference and a Weak Reference, will the Garbage Collector delete it during the next cycle?

Ans - No. As long as there is at least one Strong Reference, the object is 'Strongly Reachable' and safe from the GC.